Tutorial : Polynomial Interpolation

Part 1 : Polynomial Interpolation of GDP Data

In economics, many important quantities such as Gross Domestic Product (GDP), inflation, or population are measured at discrete time points, typically once per year or every few years. However, for analysis, forecasting, or decision-making, it is often necessary to estimate these quantities at intermediate times where no direct measurements are available.

In this practical session, we consider the problem of reconstructing the evolution of a country's GDP over time from a limited number of observations. Given a set of data points corresponding to GDP values at specific years, our goal is to build a continuous mathematical model that approximates the underlying economic trend.

Polynomial interpolation provides a natural framework for this task. Given a set of points (x_i, f_i), interpolation consists of finding a polynomial $P_n(x)$ of degree at most n such that :

$$P_n(x_i) = f_i, \quad \forall i = 0, \dots, n.$$

This polynomial can then be used to estimate GDP at any intermediate year and to analyze the behavior of the economic growth.

Several approaches exist to construct such an interpolating polynomial. In this work, we will focus on three classical methods :

— The **monomial form**, based on solving a linear system involving a Vandermonde matrix,

— The **Lagrange form**, which provides an explicit formula for the interpolating polynomial,

— The **Newton form**, based on divided differences and a recursive construction.

Although these methods are mathematically equivalent and produce the same polynomial, their numerical behavior and computational cost can differ significantly. One of the main objectives of this practical session is to compare these methods in terms of efficiency and stability when applied to real-world data.

We consider the evolution of a country's Gross Domestic Product (GDP) over time, given by the following dataset (obtained from the World Bank Open Data platform : `https://data.worldbank.org/indicator/NY.GDP.MKTP.CD`). The values have been rounded for simplicity and pedagogical clarity.

Year	GDP (trillion USD)
1980	11.214357
1990	22.943763
2000	33.621027
2010	66.036212

Table 1 – World GDP (rounded values)

Exercise 1 (Monomial Form)**.** *Assume that we are given $n+1$ distinct points $(x_0, f_0), \dots, (x_n, f_n)$. We seek a polynomial $P_n(x)$ of degree at most n in the form :*

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

such that :

$$P_n(x_i) = f_i, \quad \forall i = 0, \dots, n.$$

This leads to the following linear system :

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} f_0 \\ f_1 \\ \vdots \\ f_n \end{pmatrix}.$$

1. *Write a function that constructs the Vandermonde matrix associated with the given points, and then solve the linear system to obtain the coefficients of the interpolating polynomial, using the following algorithm (You can use, for instance,* numpy.linalg.solve *to solve the linear system) :*

Input : *data points* $(x_0, f_0), \dots, (x_n, f_n)$
Output: $(a_0, \dots, a_n)$ *coefficients of the interpolating polynomial*

```
// Construct Vandermonde matrix
```
for $i = 0, \dots, n$ **do**
  for $j = 0, \dots, n$ **do**
    $V_{i,j} \leftarrow x_i^j$
  end
end

```
// Construct right-hand side vector
```
for $i = 0, \dots, n$ **do**
  $b_i \leftarrow f_i$
end

```
// Solve linear system
```
Solve $Va = b$ *for* $a = (a_0, \dots, a_n)^T$

return $(a_0, \dots, a_n)$

Algorithm 1: Monomial Form via Vandermonde System

2. *To efficiently evaluate the polynomial* $P_n(x)$ *at a point* t*, we use Horner's algorithm :*

Input : $x \in \mathbb{R}$, *coefficients* $a_0, \cdots, a_n$
Output: $P(x)$, *where* $P(x) = a_0 + a_1 x + \cdots + a_n x^n$

```
// Initialize with highest-degree coefficient
```
$p \leftarrow a_n$

```
// Evaluate polynomial using nested multiplication
```
for $i = n-1, \cdots, 0$ **do**
  // $p = a_i + x(a_{i+1} + x(\cdots))$
  $p \leftarrow p * x + a_i$
end

```
// Return evaluated polynomial value
```
return p

Algorithm 2: Horner Algorithm

Write a function that evaluates the polynomial at a given point t *using Horner's algorithm.*

3. *Using the dataset in Table 1, construct the interpolating polynomial corresponding to these data, and estimate the GDP value in the year 1992. The exact value is* ≈ 25.465014.

4. *Plot the interpolating polynomial together with the given data points.*

Year	GDP (trillion USD)
1960	1.366676
1965	1.995647
1970	3.409880
1975	5.916540
1980	11.214357
1985	12.526402
1990	22.943763
1995	29.678994
2000	33.621027
2005	47.732965
2010	66.036212
2015	75.187709
2020	84.906994
2025	105.000000

TABLE 2 – Extended GDP dataset

5. *Estimate again the GDP value in the year 1992, using now the dataset in Table 2. What do you observe ?*

Exercise 2 (Lagrange Form)**.** *Given $n+1$ distinct points $(x_0, f_0), \ldots, (x_n, f_n)$, the interpolating polynomial $P_n(x)$ can be written in the Lagrange form :*

$$P_n(x) = \sum_{j=0}^{n} f_j L_j(x),$$

where the Lagrange basis polynomials $L_j(x)$ are defined by :

$$L_j(x) = \prod_{\substack{0 \le k \le n \\ k \ne j}} \frac{x - x_k}{x_j - x_k}.$$

1. *Write a function that evaluates the interpolating polynomial at a given point x using the Lagrange formula, according to the following algorithm :*

Input : $x \in \mathbb{R}$, *data points* $(x_0, f_0), \ldots, (x_n, f_n)$
Output: $P(x)$, *the interpolating polynomial of degree* n

```
// Initialize polynomial value
p ← 0
for i = 0, ..., n do
    // Compute j-th Lagrange basis polynomial L_j(x)
    L ← 1
    for k = 0, ..., n, k ≠ j do
        L ← L * (x − x_k)/(x_j − x_k)
    end
    // Accumulate contribution of node (x_j, f_j)
    p ← p + f_j * L
end
// Return interpolated value
return p
```

Algorithm 3: Lagrange Interpolating Polynomial

2. *Using the dataset in Table 1, evaluate the interpolating polynomial and estimate the GDP value in the year 1992.*
3. *Plot the interpolating polynomial together with the given data points.*
4. *Compare the result obtained with that of Exercise 1. Do the two methods give the same values ? Explain.*
5. *Repeat the estimation using the dataset in Table 2. Compare the results with those obtained using the monomial form.*

Exercise 3 (Newton Form)**.** *Given $n+1$ distinct points $(x_0, f_0), \ldots, (x_n, f_n)$, the interpolating polynomial can be written in the Newton form :*

$$P_n(x) = d_0 + d_1(x - x_0) + d_2(x - x_0)(x - x_1) + \cdots + d_n(x - x_0)\cdots(x - x_{n-1}),$$

where the coefficients $d_0, \ldots, d_n$ are the divided differences.

1. *Write a function that computes the divided differences using the following algorithm :*

Input : *data points* $(x_0, f_0), \cdots, (x_n, f_n)$
Output: *divided differences* $d_0, d_1, \cdots, d_n$

```
// Initialize with function values (0-th order differences)
```
for $i = 0, \cdots, n$ **do**
  $d_i \leftarrow f_i$
end

```
// Compute higher-order divided differences in-place
```
for $i = 1, \cdots, n$ **do**
  `// Update from right to left to avoid overwriting needed values`
  for $j = n, n-1, \cdots, i$ **do**
    // $d_j = f[x_{j-i}, \ldots, x_j]$
    $d_j \leftarrow \dfrac{d_j - d_{j-1}}{x_j - x_{j-i}}$
  end
end

```
// Resulting array stores Newton coefficients
```
return $d_0, d_1, \cdots, d_n$

Algorithm 4: Newton's Divided Differences

2. *Using the computed divided differences, write a function that evaluates the polynomial $P_n(x)$ at a given point t.*

   Hint : Use a Horner-like scheme adapted to the Newton form.
3. *Using the dataset in Table 2, estimate the GDP value in the year 1992.*

Part 2 : Runge Phenomenon (optional) :

Exercise 4. *We consider the approximation of the Runge function defined on the interval $[-1, 1]$:*

$$f(x) = \frac{1}{1 + 25x^2}.$$

The objective of this exercise is to study polynomial interpolation and illustrate the Runge phenomenon.

1. ***Interpolation with equidistant points :*** *Let $n \geq 1$. We consider the equidistant points :*

$$x_i = -1 + \frac{2i}{n}, \quad i = 0, \ldots, n.$$

(a) *Construct the interpolating polynomial $P_n(x)$ of f using these points.*

(b) *Plot on the same figure :*
   — *the function $f(x)$,*
   — *the interpolating polynomial $P_n(x)$,*
   for several values of n (for example $n = 5, 10, 15, 20$).

2. ***Error analysis :*** *Compute the maximum error :*

$$\|E_n\|_\infty = \max_{x\in[-1,1]} |f(x) - P_n(x)|.$$

   What do you observe as n increases ?

3. ***Interpolation with Chebyshev points :*** *We now consider the Chebyshev points :*

$$x_i = \cos\left(\frac{2i+1}{2(n+1)}\pi\right), \quad i = 0, \ldots, n.$$

(a) *Construct the interpolating polynomial $P_n(x)$ using these points.*

(b) *Plot on the same figure :*
   — *the function $f(x)$,*
   — *the interpolating polynomial,*
   for the same values of n.

4. ***Error comparison :*** *Compute again the maximum error $\|E_n\|_\infty$. Compare the errors obtained with equidistant points and Chebyshev points.*

5. *Explain why the interpolation using Chebyshev points gives better results than with equidistant points.*